

操作系统实验报告

Lab8:文件系统

信息安全

2313781 李胜林

2312796 张肇秋

2312323 杨中秀

一、实验目的

1. 了解文件系统抽象层-VFS的设计与实现
2. 了解基于索引节点组织方式的Simple FS文件系统与操作的设计与实现
3. 了解“一切皆为文件”思想的设备文件设计
4. 了解简单系统终端的实现

二、实验内容

本次实验涉及的是文件系统，通过分析了解ucore文件系统的总体架构设计，完善读写文件操作(即实现sfs_io_nolock()函数)，从新实现基于文件系统的执行程序机制（即实现load_icode()函数），从而可以完成执行存储在磁盘上的文件和实现文件读写等功能。实验八增加了文件系统，并因此实现了通过文件系统来加载可执行文件到内存中运行的功能，导致对进程管理相关的实现比较大的调整。

三、练习

(一) 练习0：填写已有实验

本实验依赖实验2/3/4/5/6/7。请把你做的实验2/3/4/5/6/7的代码填入本实验中代码中有“LAB2”/“LAB3”/“LAB4”/“LAB5”/“LAB6”/“LAB7”的注释相应部分。并确保编译通过。注意：为了能够正确执行lab8的测试应用程序，可能需对已完成的实验2/3/4/5/6/7的代码进行进一步改进。

(二) 练习1：完成读文件操作的实现：

首先了解打开文件的处理流程，然后参考本实验后续的文件读写操作的过程分析，填写在kern/fs/sfs/sfs_inode.c 中的 sfs_io_nolock() 函数，实现读文件中数据的代码。

(三) 练习2：完成基于文件系统的执行程序机制的实现：

改写 proc.c 中的 load_icode 函数和其他相关函数，实现基于文件系统的执行程序机制。执行：make qemu。如果能看看到sh用户程序的执行界面，则基本成功了。如果在sh用户界面上可以执行“ls”、“hello”等其他放置在sfs文件系统下的其他执行程序，则可以认为本实验基本成功。

(四) 扩展练习 Challenge1：完成基于“UNIX的PIPE机制”的设计方案

如果要在ucore里加入UNIX的管道（Pipe）机制，至少需要定义哪些数据结构和接口？（接口给出语义即可，不必具体实现。数据结构的设计应当给出一个(或多个) 具体的C语言struct定义。

(五) 扩展练习 Challenge2: 完成基于“UNIX的软连接和硬连接机制”的设计方案

如果要在ucore里加入UNIX的软连接和硬连接机制，至少需要定义哪些数据结构和接口？（接口给出语义即可，不必具体实现。数据结构的设计应当给出一个(或多个) 具体的C语言struct定义。

四、练习解答

(一) 练习0: 填写已有实验

这个部分由于先前lab6、lab7不是必做的实验，因此这个实验代码中我发现其实已经帮我们填写好了对应的内容了，我们只需要填写后续的lab8的练习就可以了，在此就不粘贴所有的代码了。

(二) 练习1: 完成读文件操作的实现

1.文件的处理流程

用户态进行函数封装陷入内核态

首先我们在 `file.c` 里找到函数 `open`。在这里，用户态对 `sys_open` 函数进行了封装，首先用户态调用一个 `open` 函数，这个函数被用来调用 `sys_open`

```
1 int
2 open(const char *path, uint32_t open_flags) {
3     return sys_open(path, open_flags);
4 }
```

然后我们会在 `syscall.c` 里找到 `sys_open` 函数，`sys_open` 函数也是对系统调用 `sys_call` 的封装。通过调用 `sys_open` 来调用 `sys_call`，进入内核态

```
1 int
2 sys_open(const char *path, uint64_t open_flags) {
3     return syscall(SYS_open, path, open_flags);
4 }
```

内核态通过系统调用分发对应的处理函数

我们在 `syscall.c` 里面会找到这个系统调用表：

```
1 static int (*syscalls[])(uint64_t arg[]) = {
2     [SYS_exit] sys_exit,
3     [SYS_fork] sys_fork,
4     [SYS_wait] sys_wait,
5     [SYS_exec] sys_exec,
6     [SYS_yield] sys_yield,
7     [SYS_kill] sys_kill,
8     [SYS_getpid] sys_getpid,
9     [SYS_putc] sys_putc,
10    [SYS_pgdir] sys_pgdir,
11    [SYS_gettime] sys_gettime,
12    [SYS_lab6_set_priority] sys_lab6_set_priority,
13    [SYS_sleep] sys_sleep,
14    [SYS_open] sys_open,
```

```

15     [SYS_close] sys_close,
16     [SYS_read] sys_read,
17     [SYS_write] sys_write,
18     [SYS_seek] sys_seek,
19     [SYS_fstat] sys_fstat,
20     [SYS_fsync] sys_fsync,
21     [SYS_getcwd] sys_getcwd,
22     [SYS_getdirentry] sys_getdirentry,
23     [SYS_dup] sys_dup,
24 };

```

这应该也是一个函数指针，可以通过不同的参数，映射到不同的处理函数中。然后我们就知道了在用户态调用 `syscall` 陷入内核态后，会通过这个函数指针来进行不同的处理。在本次实验中，我们是进行文件读取的操作，我们然后去寻找对应的处理函数——`sys_open`，如下所示：

```

1  static int
2  sys_open(uint64_t arg[])
3  {
4      const char *path = (const char *)arg[0];
5      uint32_t open_flags = (uint32_t)arg[1];
6      return sysfile_open(path, open_flags);
7  }

```

这里我们找到的是这个 `sys_open` 函数进行了一层拆装，调用了函数 `sysfile_open`，然后我们去找函数 `sysfile_open`，如下所示：

```

1  int
2  sysfile_open(const char *__path, uint32_t open_flags) {
3      int ret;
4      char *path;
5      if ((ret = copy_path(&path, __path)) != 0) {
6          return ret;
7      }
8      ret = file_open(path, open_flags);
9      kfree(path);
10     return ret;
11 }

```

这里这个函数首先调用了 `copy_path` 函数，用于将用户态指针 `__path` 指向的字符串复制到内核空间，防止用户态程序在系统调用过程中修改该字符串，导致内核态访问无效地址或数据错误。然后调用函数 `file_open`，随后释放内核字符串 `__path`。

这里又进行了函数调用，我们去寻找这个函数，如下所示：

```

1  int
2  file_open(char *path, uint32_t open_flags) {
3      bool readable = 0, writable = 0;
4      switch (open_flags & O_ACCMODE) {
5          case O_RDONLY: readable = 1; break;
6          case O_WRONLY: writable = 1; break;
7          case O_RDWR:
8              readable = writable = 1;
9              break;

```

```

10     default:
11         return -E_INVAL;
12     }
13     int ret;
14     struct file *file;
15     if ((ret = fd_array_alloc(NO_FD, &file)) != 0) {
16         return ret;
17     }
18     struct inode *node;
19     if ((ret = vfs_open(path, open_flags, &node)) != 0) {
20         fd_array_free(file);
21         return ret;
22     }
23     file->pos = 0;
24     if (open_flags & O_APPEND) {
25         struct stat __stat, *stat = &__stat;
26         if ((ret = vop_fstat(node, stat)) != 0) {
27             vfs_close(node);
28             fd_array_free(file);
29             return ret;
30         }
31         file->pos = stat->st_size;
32     }
33     file->node = node;
34     file->readable = readable;
35     file->writable = writable;
36     fd_array_open(file);
37     return file->fd;
38 }

```

这一部分就是首先根据 `open_flags & O_ACCMODE` 判断文件是否可读/可写，如果是非法组合，则直接返回 `-E_INVAL`。随后通过 `fd_array_alloc(NO_FD, &file)` 在当前进程的文件描述符表中找一个空槽（`fd`）并分配 `struct file`。然后调用 `vfs_open` 获取 `inode`，这个函数我们随后会进行分析。下一步是我们需要处理 `O_APPEND` 标志，如果带有这个标志，我们需要通过调用 `vop_fstat` 来获取文件的大小，随后通过 `file->pos=stat->st_size` 进行赋值，这样我们写文件开始的标志（`file->pos`）的位置就在文件尾了，随后就可以从文件最后开始继续像后面写内容了。

我们寻找调用的函数 `vfs_open()`：

```

1  int
2  vfs_open(char *path, uint32_t open_flags, struct inode **node_store) {
3      bool can_write = 0;
4      switch (open_flags & O_ACCMODE) {
5          case O_RDONLY:
6              break;
7          case O_WRONLY:
8          case O_RDWR:
9              can_write = 1;
10             break;
11         default:
12             return -E_INVAL;
13     }
14
15     if (open_flags & O_TRUNC) {
16         if (!can_write) {

```

```

17         return -E_INVALID;
18     }
19 }
20
21 int ret;
22 struct inode *node;
23 bool excl = (open_flags & O_EXCL) != 0;
24 bool create = (open_flags & O_CREAT) != 0;
25 ret = vfs_lookup(path, &node);
26
27 if (ret != 0) {
28     if (ret == -16 && (create)) {
29         char *name;
30         struct inode *dir;
31         if ((ret = vfs_lookup_parent(path, &dir, &name)) != 0) {
32             return ret;
33         }
34         ret = vop_create(dir, name, excl, &node);
35     } else return ret;
36 } else if (excl && create) {
37     return -E_EXISTS;
38 }
39 assert(node != NULL);
40
41 if ((ret = vop_open(node, open_flags)) != 0) {
42     vop_ref_dec(node);
43     return ret;
44 }
45
46 vop_open_inc(node);
47 if (open_flags & O_TRUNC || create) {
48     if ((ret = vop_truncate(node, 0)) != 0) {
49         vop_open_dec(node);
50         vop_ref_dec(node);
51         return ret;
52     }
53 }
54 *node_store = node;
55 return 0;
56 }

```

该函数首先要检查 `O_TRUNC` (即清空文件) 的合法性, 我们在清空文件之前, 这个文件必须可写, 否则我们会抛出错误。然后会调用 `vfs_lookup` 的函数, 去查找 `inode`。如果查找成功, 那么这个 `node` 就是文件的 `inode`; 如果查找失败, 那么就必须自己创建一个。我们通过 `vfs_lookup_parent` 找到父目录的 `inode` 和文件名, 然后通过 `vop_create` 实现对应的创建功能。下一步我们调用 `vop_open` 函数, 来触发具体的文件系统对应的 `inode` open 操作, 这样就能实现文件打开的操作了, 但我们还需要具体看一下是怎么找到 `inode` 的, 我们接下来继续分析。

下一步我们要查找函数 `vfs_open` 调用的 `vfs_lookup` 和 `vfs_lookup_parent` 函数, 如下所示:

```

1 int
2 vfs_lookup(char *path, struct inode **node_store) {
3     int ret;
4     struct inode *node;

```

```

5     if ((ret = get_device(path, &path, &node)) != 0) {
6         return ret;
7     }
8     if (*path != '\0') {
9         ret = vop_lookup(node, path, node_store);
10        vop_ref_dec(node);
11        return ret;
12    }
13    *node_store = node;
14    return 0;
15 }

```

`vfs_lookup` 用于把完整路径解析成目标 `inode`。它先调用 `get_device` 解析设备前缀并返回起始 `inode`，同时把 `path` 指针推进到设备名之后的剩余路径。若 `get_device` 失败直接返回错误码。随后判断 `path` 是否为 `'\0'`：若还有子路径，则调用 `vop_lookup` 在该起始 `inode` 下继续查找最终文件目录，并在查找结束后通过 `vop_ref_dec` 减少起始 `inode` 引用计数。若剩余路径为空，说明路径只指向设备根或起始 `inode` 本身，直接把 `node` 赋给 `node_store` 并返回0。

```

1  int
2  vfs_lookup_parent(char *path, struct inode **node_store, char **endp){
3      int ret;
4      struct inode *node;
5      if ((ret = get_device(path, &path, &node)) != 0) {
6          return ret;
7      }
8      *endp = path;
9      *node_store = node;
10     return 0;
11 }

```

`vfs_lookup_parent` 用于创建场景下的父目录查找，它同样先用 `get_device` 解析设备并获得起始 `inode`，同时把 `path` 更新为设备名后的剩余部分。与 `vfs_lookup` 不同，它不继续向下解析目录层级，而是把 `endp` 直接指向当前 `path` 作为后续解析末尾文件名的起点，并把起始 `inode` 返回给 `node_store`。上层在需要 `O_CREAT` 时会基于该父目录 `inode` 以及 `endp` 指向的路径部分提取最终 `name`，再调用 `vop_create` 在父目录下创建目标。

下一步我们寻找 `vop_lookup` 函数，如下所示：

```

1  #define vop_lookup(node, path, node_store)    (__vop_op(node, lookup)(node,
        path, node_store))

```

所以其实 `vop_lookup` 就是 `__vop_op` 根据不同的指令分发处理，在这里，它遇到的是 `lookup` 操作，就执行相应的函数，我们找到这个 `__vop_op` 的定义：

```

1  struct inode_ops {
2      unsigned long vop_magic;
3      int (*vop_open)(struct inode *node, uint32_t open_flags);
4      int (*vop_close)(struct inode *node);
5      int (*vop_read)(struct inode *node, struct iobuf *iob);
6      int (*vop_write)(struct inode *node, struct iobuf *iob);
7      int (*vop_fstat)(struct inode *node, struct stat *stat);
8      int (*vop_fsync)(struct inode *node);
9      int (*vop_namefile)(struct inode *node, struct iobuf *iob);

```

```

10     int (*vop_getdirent)(struct inode *node, struct iobuf *iob);
11     int (*vop_reclaim)(struct inode *node);
12     int (*vop_gettype)(struct inode *node, uint32_t *type_store);
13     int (*vop_tryseek)(struct inode *node, off_t pos);
14     int (*vop_truncate)(struct inode *node, off_t len);
15     int (*vop_create)(struct inode *node, const char *name, bool excl,
struct inode **node_store);
16     int (*vop_lookup)(struct inode *node, char *path, struct inode
**node_store);
17     int (*vop_ioctl)(struct inode *node, int op, void *data);
18 };
19
20 #define __vop_op(node, sym)
21     (
22         struct inode *__node = (node);
23         assert(__node != NULL && __node->in_ops != NULL && __node->in_ops-
>vop_##sym != NULL);
24         inode_check(__node, #sym);
25         __node->in_ops->vop_##sym;
26     )

```

找到这一步我们继续，我们会发现在 `sfs_inode.c` 中定义了SFS 的 inode 操作函数表。对于目录类型的 inode，其操作函数表是 `sfs_node_dirops`：

```

1  static const struct inode_ops sfs_node_dirops = {
2      .vop_magic          = VOP_MAGIC,
3      .vop_open           = sfs_opendir,
4      .vop_close          = sfs_close,
5      .vop_fstat          = sfs_fstat,
6      .vop_fsync          = sfs_fsync,
7      .vop_namefile       = sfs_namefile,
8      .vop_getdirent      = sfs_getdirent,
9      .vop_reclaim        = sfs_reclaim,
10     .vop_gettype         = sfs_gettype,
11     .vop_lookup          = sfs_lookup,
12 };

```

因此当VFS调用 `vop_lookup` 时，实际上执行的是 `sfs_lookup`，我们找到 `sfs_lookup` 函数，如下所示：

```

1  static int
2  sfs_lookup(struct inode *node, char *path, struct inode **node_store) {
3      struct sfs_fs *sfs = fsop_info(vop_fs(node), sfs);
4      // 确保路径有效且不是根目录（根目录不需要查找）
5      assert(*path != '\0' && *path != '/');
6      vop_ref_inc(node); // 增加当前目录 inode 的引用计数
7      struct sfs_inode *sin = vop_info(node, sfs_inode);
8
9      // 只有目录才能执行 lookup 操作

```

```

10     if (sin->din->type != SFS_TYPE_DIR) {
11         vop_ref_dec(node);
12         return -E_NOTDIR;
13     }
14
15     struct inode *subnode;
16     // 调用核心查找函数 sfs_lookup_once
17     int ret = sfs_lookup_once(sfs, sin, path, &subnode, NULL);
18
19     vop_ref_dec(node);
20     if (ret != 0) {
21         return ret;
22     }
23     *node_store = subnode; // 返回找到的 inode
24     return 0;
25 }

```

这个函数主要做了两件事，首先是调用 `sfs_dirent_search_nolock` 获取文件名对应的磁盘 inode 编号，然后再通过调用 `sfs_load_inode` 将该磁盘 inode 加载为内存中的 `struct inode`。

然后我去寻找调用的 `sfs_dirent_search_nolock`，如下所示：

```

1  static int
2  sfs_dirent_search_nolock(struct sfs_fs *sfs, struct sfs_inode *sin, const
3  char *name, uint32_t *ino_store, int *slot, int *empty_slot) {
4      assert(strlen(name) <= SFS_MAX_FNAME_LEN);
5      struct sfs_disk_entry *entry;
6      if ((entry = kmalloc(sizeof(struct sfs_disk_entry))) == NULL) {
7          return -E_NO_MEM;
8      }
9      int ret, i, nslots = sin->din->blocks; // 目录占用的块数（简化实现中，一个块一个目录项）
10
11     // 遍历目录的所有数据块
12     for (i = 0; i < nslots; i++) {
13         // 读取第 i 个目录项
14         if ((ret = sfs_dirent_read_nolock(sfs, sin, i, entry)) != 0) {
15             goto out;
16         }
17         if (entry->ino == 0) {
18             continue; // 空闲项
19         }
20         // 比较文件名
21         if (strcmp(name, entry->name) == 0) {
22             // 找到匹配项，返回索引和 inode 编号
23             if (slot != NULL) *slot = i;
24             *ino_store = entry->ino;
25             goto out;
26         }
27     }
28     ret = -E_NOENT; // 未找到
29 out:
30     kfree(entry);
31     return ret;

```


该函数遍历目录文件的所有数据块，读取其中的 `sfs_disk_entry` 结构，并使用 `strcmp` 比较文件名。如果匹配成功，就返回该文件对应的 `inode` 编号。这样我们就找到 `inode` 了，我们的 `vfs_open` 函数的逻辑也就通了。

总述

通过上述分析，我们可以知道文件的处理总流程是：

首先我们在用户态调用 `open` -> `sys_open` -> `syscall`，然后陷入内核态后，依次执行 `sys_open` -> `sysfile_open` -> `file_open` -> `vfs_open` -> `vfs_lookup`，最后通过 `sfs` 执行 `vop_lookup` -> `sfs_lookup` -> `sfs_lookup_once` -> `sfs_dirent_search_nolock`，来读取磁盘块并比较文件名。当我们找到 `inode` 后，`vfs_open` 会继续调用 `vop_open`（即 `sys_openfile`），完成文件的打开操作。

2.填写 `sfs_io_nolock()` 函数

```

1  if ((blkoff = offset % SFS_BLKSIZE) != 0) {
2      size = (nblks != 0) ? (SFS_BLKSIZE - blkoff) : (endpos - offset);
3      if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
4          goto out;
5      }
6      if ((ret = sfs_buf_op(sfs, buf, size, ino, blkoff)) != 0) {
7          goto out;
8      }
9      alen += size;
10     if (nblks == 0) {
11         goto out;
12     }
13     buf = (char *)buf + size;
14     blkno ++;
15     nblks --;
16 }
17
18 size = SFS_BLKSIZE;
19 while (nblks != 0) {
20     if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
21         goto out;
22     }
23     if ((ret = sfs_block_op(sfs, buf, ino, 1)) != 0) {
24         goto out;
25     }
26     alen += size;
27     buf = (char *)buf + size;
28     blkno ++;
29     nblks --;
30 }
31
32 if ((size = endpos % SFS_BLKSIZE) != 0) {
33     if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
34         goto out;
35     }
36     if ((ret = sfs_buf_op(sfs, buf, size, ino, 0)) != 0) {
37         goto out;
38     }

```

```

39 |     alen += size;
40 | }

```

这个函数是 `SFS` 文件系统中进行文件读写核心底层实现，实现了将磁盘上的数据读入内存缓冲区，或者将内存缓冲区的数据写入磁盘的逻辑。在实现中，我们需要首先检查读写操作的起始位置是否在磁盘块的边界上，如果未对齐，则先计算出第一块中需要处理的数据大小，获取对应的磁盘块号并进行部分读写。接着，对于中间跨越的完整磁盘块，代码通过循环直接以块为单位进行读写操作，这样可以显著提高数据传输的效率。最后，代码处理结束位置未对齐的情况，将最后一块中剩余的数据进行读写。整个过程通过调用 `sfs_bmap_load_nolock` 获取物理块号，并结合 `sfs_buf_op` 或 `sfs_block_op` 完成了数据在内存缓冲区和磁盘块之间的正确传输。

(三) 练习2：完成基于文件系统的执行程序机制的实现

我们只需要改写这几个函数就可以了，分别是 `alloc_proc`、`proc_run`、`do_fork` 和 `load_icode`。

`alloc_proc`

这个函数我们就在原来的基础上加了一行，如下所示：

```

1 | proc->filesp = NULL;

```

这是进程控制块中用于管理打开文件的结构，初始化为空。

`proc_run`

这里我们加了一行，如下所示：

```

1 | if (proc->pgdir != 0) {
2 |     lsatp(proc->pgdir); // load pdir 2 satp of new proc
3 | } else {
4 |     lsatp(boot_pgdir_pa); // pgdir of kernel t
5 | }
6 | flush_tlb(); // 新增
7 | switch_to(&(prev->context), &(next->context));

```

在调用 `lsatp` 切换页表后，添加了 `flush_tlb`。这是为了确保 TLB 中的旧映射被清除，CPU 能正确使用新的地址空间。

`do_fork`

在原来的框架上添加了下面这个代码：

```

1 | if (copy_files(clone_flags, proc) != 0)
2 | { // for LAB8
3 |     goto bad_fork_cleanup_kstack;
4 | }
5 |
6 | bad_fork_cleanup_fs: // for LAB8
7 |     put_files(proc);

```

我们增加了 `copy_files` 调用，确保子进程在创建时能够正确继承或复制父进程打开的文件描述符表。

load_icode

主要修改的是这一部分，如下所示：

```
1 static int
2 load_icode(int fd, int argc, char **kargv)
3 {
4     assert(argc >= 0 && argc <= EXEC_MAX_ARG_NUM);
5     if (current->mm != NULL) {
6         panic("load_icode: current->mm must be empty.\n");
7     }
8     int ret = -E_NO_MEM;
9     struct mm_struct *mm;
10    if ((mm = mm_create()) == NULL) {
11        goto bad_mm;
12    }
13    if (setup_pgdir(mm) != 0) {
14        goto bad_pgdir_cleanup_mm;
15    }
16    struct Page *page;
17    struct elfhdr __elf, *elf = &__elf;
18    if ((ret = load_icode_read(fd, elf, sizeof(struct elfhdr), 0)) != 0) {
19        goto bad_elf_cleanup_pgdir;
20    }
21    if (elf->e_magic != ELF_MAGIC) {
22        ret = -E_INVALID_ELF;
23        goto bad_elf_cleanup_pgdir;
24    }
25    struct proghdr __ph, *ph = &__ph;
26    uint32_t vm_flags, perm;
27    for (int i = 0; i < elf->e_phnum; i++) {
28        off_t phoff = elf->e_phoff + sizeof(struct proghdr) * i;
29        if ((ret = load_icode_read(fd, ph, sizeof(struct proghdr), phoff))
30            != 0) {
31            goto bad_cleanup_mmap;
32        }
33        if (ph->p_type != ELF_PT_LOAD) {
34            continue;
35        }
36        if (ph->p_filesz > ph->p_memsz) {
37            ret = -E_INVALID_ELF;
38            goto bad_cleanup_mmap;
39        }
40        if (ph->p_filesz == 0) {
41            // continue;
42        }
43        vm_flags = 0, perm = PTE_U | PTE_V;
44        if (ph->p_flags & ELF_PF_X) vm_flags |= VM_EXEC;
45        if (ph->p_flags & ELF_PF_W) vm_flags |= VM_WRITE;
46        if (ph->p_flags & ELF_PF_R) vm_flags |= VM_READ;
47        if (vm_flags & VM_READ) perm |= PTE_R;
48        if (vm_flags & VM_WRITE) perm |= PTE_W;
49        if (vm_flags & VM_EXEC) perm |= PTE_X;
50        if ((ret = mm_map(mm, ph->p_va, ph->p_memsz, vm_flags, NULL)) != 0)
51            goto bad_cleanup_mmap;
52    }
53    return ret;
54 }
```

```

50         goto bad_cleanup_mmap;
51     }
52     off_t offset = ph->p_offset;
53     size_t len = ph->p_filesz;
54     uintptr_t va = ph->p_va;
55     uintptr_t end_va = va + len;
56     uintptr_t start = ROUNDDOWN(va, PGSIZE);
57     uintptr_t end = ROUNDUP(va + ph->p_memsz, PGSIZE);
58     uintptr_t la = start;
59     while (la < end) {
60         if ((page = pgdir_alloc_page(mm->pgdir, la, perm)) == NULL) {
61             ret = -E_NO_MEM;
62             goto bad_cleanup_mmap;
63         }
64
65         void *kva = page2kva(page);
66         uintptr_t page_start = la;
67         uintptr_t page_end = la + PGSIZE;
68         uintptr_t valid_start = (page_start > va) ? page_start : va;
69         uintptr_t valid_end = (page_end < end_va) ? page_end : end_va;
70
71         if (valid_start < valid_end) {
72             if ((ret = load_icode_read(fd, kva + (valid_start -
73 page_start), valid_end - valid_start, offset + (valid_start - va))) != 0) {
74                 goto bad_cleanup_mmap;
75             }
76             if (valid_start > page_start) {
77                 memset(kva, 0, valid_start - page_start);
78             }
79             if (valid_end < page_end) {
80                 memset(kva + (valid_end - page_start), 0, page_end -
81 valid_end);
82             }
83             } else {
84                 memset(kva, 0, PGSIZE);
85             }
86             la += PGSIZE;
87         }
88     }
89     vm_flags = VM_READ | VM_WRITE | VM_STACK;
90     if ((ret = mm_map(mm, USTACKTOP - USTACKSIZE, USTACKSIZE, vm_flags,
91 NULL)) != 0) {
92         goto bad_cleanup_mmap;
93     }
94     assert(pgdir_alloc_page(mm->pgdir, USTACKTOP-PGSIZE , PTE_USER) !=
95 NULL);
96     assert(pgdir_alloc_page(mm->pgdir, USTACKTOP-2*PGSIZE , PTE_USER) !=
97 NULL);
98     assert(pgdir_alloc_page(mm->pgdir, USTACKTOP-3*PGSIZE , PTE_USER) !=
99 NULL);
100    assert(pgdir_alloc_page(mm->pgdir, USTACKTOP-4*PGSIZE , PTE_USER) !=
101 NULL);
102    page = get_page(mm->pgdir, USTACKTOP - PGSIZE, NULL);
103    char *kstacktop = page2kva(page) + PGSIZE;
104    char *head = kstacktop;
105    char* uargv_ptr[EXEC_MAX_ARG_NUM];

```

```

98     for (int i = 0; i < argc; i++) {
99         int len = strlen(kargv[i], EXEC_MAX_ARG_LEN + 1) + 1;
100         head -= len;
101         memcpy(head, kargv[i], len);
102         uargv_ptr[i] = (char*)(USTACKTOP - (kstacktop - head));
103     }
104     head = (char*)ROUNDDOWN((uintptr_t)head, sizeof(uintptr_t));
105     uintptr_t *argv_array = (uintptr_t *) (head - (argc + 1) *
sizeof(uintptr_t));
106     for (int i = 0; i < argc; i++) {
107         argv_array[i] = (uintptr_t)uargv_ptr[i];
108     }
109     argv_array[argc] = 0;
110     head = (char*)argv_array;
111     head -= sizeof(int);
112     *(int*)head = argc;
113     uintptr_t user_sp = USTACKTOP - (kstacktop - head);
114     mm_count_inc(mm);
115     current->mm = mm;
116     current->pgdir = PADDR(mm->pgdir);
117     lsatp(current->pgdir);
118     struct trapframe *tf = current->tf;
119     memset(tf, 0, sizeof(struct trapframe));
120     tf->gpr.sp = user_sp;
121     tf->epc = elf->e_entry;
122     tf->status = read_csr(sstatus) | SSTATUS_SPIE;
123     tf->status &= ~SSTATUS_SPP;
124     return 0;
125
126 bad_cleanup_mmap:
127     exit_mmap(mm);
128 bad_elf_cleanup_pgdir:
129     put_pgdir(mm);
130 bad_pgdir_cleanup_mm:
131     mm_destroy(mm);
132 bad_mm:
133     return ret;
134 }

```

我们的核心目标是把一个ELF可执行文件正确放入进程地址空间。首先，在开始阶段创建新的内存管理结构并初始化页目录，读取ELF头与程序头信息，对类型为可加载的段建立虚拟内存映射。然后逐页分配物理内存，将文件中的指令和数据拷贝到对应位置，同时对未在文件中出现但需要占用内存的区域进行清零处理。接着在高地址处分配用户栈，将参数字符串压入栈内并构造 `argc` 与 `argv` 布局。最后设置页表基址与陷阱帧，初始化用户态栈指针和入口地址，使进程具备从用户态开始执行的完整上下文。

(四) 扩展练习 Challenge1: 完成基于“UNIX的PIPE机制”的设计方案

1.概要设计

UNIX 管道 (Pipe) 是一种进程间通信机制，允许一个进程的输出作为另一个进程的输入。在 ucore 中实现管道，本质上是创建一个内核缓冲区，并将其抽象为文件 (inode)，使得进程可以通过文件描述符 (fd) 对其进行读写。管道是单向的字节流。通常 `pipe()` 系统调用会返回两个文件描述符，一个用于读，一个用于写。

2. 数据结构设计

我们需要定义一个管道特有的信息结构体，它可以作为 `inode` 的 `in_info` 联合体的一部分，或者作为一个独立的结构体被 `inode` 引用。

```
1  #define PIPE_SIZE 4096
2
3  /* 管道缓冲区及控制信息 */
4  struct pipe_info {
5      char buffer[PIPE_SIZE];    // 环形缓冲区
6      off_t read_pos;            // 读指针位置
7      off_t write_pos;           // 写指针位置
8      size_t cnt;                // 当前缓冲区中的字节数
9
10     semaphore_t mutex;          // 互斥锁，保护缓冲区操作
11     semaphore_t wait_reader;    // 读者等待信号量（用于同步：缓冲区空时等待）
12     semaphore_t wait_writer;   // 写者等待信号量（用于同步：缓冲区满时等待）
13
14     int readers;                // 打开读端的进程数
15     int writers;               // 打开写端的进程数
16     bool is_closed;            // 管道是否已完全关闭
17 };
18
19 /* 扩展 inode 的 union，或者在创建 inode 时分配 private data */
20 // 在 ucore 的 inode 结构中，通常通过 device 或 sfs_inode 来区分。
21 // 对于 pipe，我们可以定义一个新的 inode type: inode_type_pipe_info
```

3. 接口及语义

`int pipe(int fd[2])`

- **语义：**创建一个管道，分配两个文件描述符。 `fd[0]` 用于读， `fd[1]` 用于写。
- **实现思路：**
 1. 分配一个新的 `struct pipe_info`。
 2. 初始化缓冲区、信号量（`mutex=1`, `wait_reader=0`, `wait_writer=PIPE_SIZE`）。
 3. 创建两个 `struct file` 对象，分别关联到同一个（或两个关联的） `inode`。
 4. 这两个 `file` 对象分别标记为只读和只写。
 5. 将 `file` 对象映射到当前进程的两个空闲 `fd`。

`read(fd, buf, count)`

- **语义：**从管道读取数据。
- **实现思路：**
 1. 获取 `mutex`。
 2. 如果缓冲区为空：
 - 如果写端已关闭（`writers == 0`），返回 0 (EOF)。
 - 否则，释放 `mutex`，在 `wait_reader` 上等待（P操作），被唤醒后重新获取 `mutex`。
 3. 从 `buffer` 读取数据到用户 `buf`。
 4. 更新 `read_pos` 和 `cnt`。

- 唤醒等待的写者 (V操作 `wait_writer`) 。
- 释放 `mutex` 。

`write(fd, buf, count)`

- **语义：**向管道写入数据。
- **实现思路：**
 1. 获取 `mutex` 。
 2. 如果读端已关闭 (`readers == 0`)，发送 `SIGPIPE` 信号给进程，返回错误。
 3. 如果缓冲区已满：
 - 释放 `mutex`，在 `wait_writer` 上等待 (P操作)，被唤醒后重新获取 `mutex` 。
 4. 将数据从用户 `buf` 写入 `buffer` 。
 5. 更新 `write_pos` 和 `cnt` 。
 6. 唤醒等待的读者 (V操作 `wait_reader`) 。
 7. 释放 `mutex` 。

`close(fd)`

- **语义：**关闭管道的一端。
- **实现思路：**
 1. 获取 `mutex` 。
 2. 如果是读端关闭，`readers--`；如果是写端关闭，`writers--` 。
 3. 如果 `readers == 0` 且 `writers == 0`，释放 `pipe_info` 内存。
 4. 如果 `writers` 变为 0，唤醒所有等待的读者 (让它们读到 EOF) 。
 5. 如果 `readers` 变为 0，唤醒所有等待的写者 (让它们收到错误) 。
 6. 释放 `mutex` 。

4. 同步互斥处理

- **互斥：**使用 `mutex` 信号量保证对 `buffer`、`read_pos`、`write_pos` 等状态的原子访问。
- **同步：**
 - **读者等待：**当缓冲区空时，读者在 `wait_reader` 上等待。写者写入数据后执行 `up(&wait_reader)` 。
 - **写者等待：**当缓冲区满时，写者在 `wait_writer` 上等待。读者读出数据后执行 `up(&wait_writer)` 。

(五) 扩展练习 Challenge2：完成基于“UNIX的软连接和硬连接机制”的设计方案

1.概要设计

硬链接：在文件系统中，多个目录项 (Directory Entry) 指向同一个 inode。删除一个硬链接只是减少 inode 的引用计数，只有当引用计数为 0 且文件未被打开时，才真正删除文件数据。

软链接：一种特殊类型的文件，其内容是指向另一个文件的路径字符串。

2.数据结构设计

硬链接

SFS 的磁盘 inode 结构 `struct sfs_disk_inode` 已经包含 `nlinks` 字段，我们需要在内存 inode `struct sfs_inode` 中也维护这个计数，并确保同步即可，如下所示：

```
1  /* inode (on disk) */
2  struct sfs_disk_inode {
3      uint32_t size;                                /* size of the file (in
bytes) */
4      uint16_t type;                                /* one of SYS_TYPE_*
above */
5      uint16_t nlinks;                              /* # of hard links to
this file */
6      uint32_t blocks;                              /* # of blocks */
7      uint32_t direct[SFS_NDIRECT];                /* direct blocks */
8      uint32_t indirect;                            /* indirect blocks */
9      //      uint32_t db_indirect;                  /* double indirect
blocks */
10     //      unused
11 };
12 /* inode for sfs */
13 struct sfs_inode {
14     struct sfs_disk_inode *din;                    /* on-disk inode */
15     uint32_t ino;                                  /* inode number */
16     bool dirty;                                    /* true if inode
modified */
17     int reclaim_count;                             /* kill inode if it hits
zero */
18     semaphore_t sem;                               /* semaphore for din */
19     list_entry_t inode_link;                       /* entry for linked-list
in sfs_fs */
20     list_entry_t hash_link;                        /* entry for hash
linked-list in sfs_fs */
21     uint16_t nlinks;                               /*新增硬链接计数*/
22 };
```

软链接

软链接我们只需要定义一种新的文件类型就可以了，如下所示：

```
1 | #define SFS_TYPE_LINK    3
```

3.接口及语义

硬链接

```
int link(const char *oldpath, const char *newpath)
```

- **语义：**为 `oldpath` 指定的文件创建一个名为 `newpath` 的新硬链接。
- **实现思路：**
 1. 查找 `oldpath` 对应的 inode (`old_inode`)。
 2. 检查 `old_inode` 是否为目录（通常不允许对目录创建硬链接以避免环）。

3. 在 `newpath` 的父目录下创建一个新的目录项 (entry) , 指向 `old_inode` 的 inode 编号 (ino)。
4. `old_inode->nlinks++`。
5. 将 `old_inode` 的更新写回磁盘。

```
int unlink(const char *path)
```

- **语义:** 删除 `path` 对应的目录项。
- **实现思路:**
 1. 查找 `path` 对应的 inode (`target_inode`) 和其父目录 inode。
 2. 从父目录中删除对应的目录项。
 3. `target_inode->nlinks--`。
 4. 如果 `target_inode->nlinks == 0` 且 `open_count == 0`:
 - 释放 `target_inode` 占用的所有数据块。
 - 释放 `target_inode` 本身。
 5. 否则, 仅将 `nlinks` 的变化写回磁盘。

软链接

```
int symlink(const char *target, const char *linkpath)
```

- **语义:** 创建一个名为 `linkpath` 的软链接, 指向 `target`。
- **实现思路:**
 1. 在 `linkpath` 的父目录下创建一个新文件。
 2. 设置新文件的 inode 类型为 `SFS_TYPE_LINK`。
 3. 将 `target` 字符串写入新文件的数据块中。

```
int readlink(const char *path, char *buf, size_t bufsiz)
```

- **语义:** 读取软链接本身的内容。
- **实现思路:**
 1. 打开 `path` 对应的 inode。
 2. 检查 inode 类型是否为 `SFS_TYPE_LINK`。
 3. 读取 inode 的数据块内容到 `buf`。

`open()` 的修改

- **语义:** 当打开文件时, 如果遇到软链接, 需要根据策略决定是否跟随。
- **实现思路:**
 1. 在 `vop_lookup` 或 `vfs_lookup` 过程中, 如果解析到的 inode 是 `SFS_TYPE_LINK`:
 2. 读取其内容。
 3. 递归调用 `lookup` 解析目标路径。
 4. 需要设置最大递归深度以防止死循环。

4.同步互斥处理

硬链接

在 `nlinks` 修改时需要原子操作或锁来保护，而在SFS中已经有 `mutex_sem` 用于 `link` 和 `unlink` 操作。在 `unlink` 时，需要检查 `open_count`，这需要 VFS 层和 SFS 层的配合。如果文件被打开，`unlink` 只减少 `nlinks`，文件删除推迟到 `vop_reclaim`（即最后一个 `close`）时进行。

软链接

创建软链接涉及分配 `inode` 和数据块，需要使用 SFS 的位图锁。读取软链接内容是只读操作，可以使用读写锁或普通的 `inode` 锁。

五、实验结果

我们使用 `make qemu` 进入文件系统，如下所示：

```
yzx@yzx:~/桌面/labcode/原始代码/lab8$ make qemu
etext 0xc020b48e (virtual)
edata 0xc0291060 (virtual)
end 0xc0296910 (virtual)
Kernel executable memory footprint: 603KB
DTB Init
HartID: 0
DTB Address: 0x82200000
Physical Memory from DTB:
Base: 0x0000000080000000
Size: 0x0000000080000000 (128 MB)
End: 0x0000000087ffffff
DTB init completed
memory management: default_pmm_manager
physical memory map:
memory: 0x00000000, [0x00000000, 0x87ffffff].
vapaofset is 18446744070488326144
check_alloc_page() succeeded!
Page table directory switch succeeded!
Kernel stack guardians set succeeded!
check_pgdire() succeeded!
check_boot_pgdire() succeeded!
use SLOB allocator
kmmalloc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
sched class: RR_scheduler
Initrd: 0xc0214010 - 0xc021bd0f, size: 0x00007d00
Initrd: 0xc021bd10 - 0xc029100f, size: 0x00075300
sfs: mount: 'simple file system' (106/11/117)
vfs: mount disk0.
++ setup timer interrupts
kernel_execve: pid = 2, name = "sh".
user sh is running!!!
```

对于下面的测试样例，我们都进行了测试，如下所示：

```
yzx@yzx:~/桌面/labcode/原始代码/lab8$ make qemu
Initrd: 0xc021bd10 - 0xc029100f, size: 0x00075300
sfs: mount: 'simple file system' (106/11/117)
vfs: mount disk0.
++ setup timer interrupts
kernel_execve: pid = 2, name = "sh".
user sh is running!!!
fork ok.
badarg pass.
user panic at user/badsegment.c:9:
FAIL: T.T
error: -10 - panic failure
value is -1.
user panic at user/divzero.c:9:
FAIL: T.T
error: -10 - panic failure
error: -16 - no such file or directory
I am the parent. Forking the child...
I am parent, fork a child pid 9
I am the parent, waiting now..
I am the child.
waitpid 9 ok.
exit pass.
```

```
yzx@yzx:~/桌面/labcode/原始代码/lab8$ make qemu
0031: I am '11'
0032: I am '000'
0033: I am '001'
0034: I am '010'
0035: I am '011'
0036: I am '100'
0037: I am '101'
0038: I am '110'
0039: I am '111'
Hello world!!
I am process 50.
hello pass.
I am 60, print pgdir.
pgdir pass.
set priority to 6
main: fork ok, now need to wait pids.
set priority to 1
child pid 62, acc 4000, time 152780
set priority to 2
child pid 63, acc 4000, time 152790
set priority to 3
child pid 64, acc 4000, time 152790
set priority to 4
child pid 65, acc 4000, time 152790
set priority to 5
child pid 66, acc 4000, time 152790
main: pid 62, acc 4000, time 152790
main: pid 63, acc 4000, time 152790
main: pid 64, acc 4000, time 152790
main: pid 65, acc 4000, time 152790
main: pid 66, acc 4000, time 152800
main: wait pids over
stride sched correct result: 1 1 1 1
$
```

```
yzx@yzx:~/桌面/labcode/原始代码/lab8$ make qemu
pid 75 done!.
pid 78 done!.
pid 82 done!.
pid 84 done!.
pid 87 done!.
matrix pass.
user sh is running!!!
sleep 1 x 100 slices.
sleep 2 x 100 slices.
sleep 3 x 100 slices.
sleep 4 x 100 slices.
sleep 5 x 100 slices.
sleep 6 x 100 slices.
sleep 7 x 100 slices.
sleep 8 x 100 slices.
sleep 9 x 100 slices.
sleep 10 x 100 slices.
use 10000 msecs.
sleep pass.
sleepkill pass.
I am the parent. Forking the child...
I am the parent. Running the child...
I am the child. spinning ...
I am the parent. Killing the child...
kill returns 0
wait returns 0
spin may pass.
wait child 1.
child 1.
child 2.
kill parent ok.
error: -9 - process is killed
$ kill child1 ok.
Hello, I am process 99.
```

```
Hello, I am process 99.  
Back in process 99, iteration 0.  
Back in process 99, iteration 1.  
Back in process 99, iteration 2.  
Back in process 99, iteration 3.  
Back in process 99, iteration 4.  
All done in process 99.  
yield pass.  
I am child 0  
I am child 1  
I am child 2  
I am child 3  
I am child 4  
I am child 5  
I am child 6  
I am child 7  
I am child 8  
I am child 9  
I am child 10  
I am child 11  
I am child 12  
I am child 13  
I am child 14  
I am child 15  
I am child 16  
I am child 17  
I am child 18  
I am child 19  
I am child 20  
I am child 21  
I am child 22  
I am child 23  
I am child 24  
I am child 25  
I am child 26  
I am child 27  
I am child 28  
I am child 29  
I am child 30  
I am child 31  
forktest pass.  
$
```

六、知识点总结

经过实验和理论课程的学习，我们对文件系统有了更深层次的认知。接下来，我们将从文件系统体系结构、

文件系统加载过程、文件的打开以及一些文件系统管理策略的角度，对其知识要点进行总结。

- 文件系统体系结构：实验中的整个文件系统基本可分为五层，分别是：用户层封装的接口、虚拟文件系统层（VFS）、磁盘文件系统（SFS）、设备层（在此次实验中为disk0、stdin、stdout）、设备驱动层。其中，最顶层和最底层在文件系统设计过程中较为简单，复杂的是将不同设备中的不同文件系统接口对接到操作系统统一的虚拟文件系统层的工作。

- 1 其中，**vfs** 层是整个体系的中枢，负责屏蔽不同文件系统的差异，统一提供**inode**接口，如：**open**、**read**、**write**、**stat**、**ioctl**等函数。
- 2 在本次实验中，**vfs**使用设备表**vdev_list**对设备信息进行管理，利用**vfs_dev_t**结构中的**fs**字段，可以记录设备上的文件系统挂载信息。
- 3 在其下的**inode**索引节点定义中，使用**in_fs**字段记录其所属的文件系统，并使用**in_ops**字段记录其操作函数数组，用于后续向下对接接口。
- 4
- 5 **sfs**层设计了一个具体的文件系统，在此实验中，它将被挂载到**disk0**设备上，总体来说，其使用超级块、根目录、位示图等方式，对**sfs**的信息进行了管理。
- 6 其首要工作是提供一个**sfs_do_mount**接口，其在**vfs_mount**中作为函数指针参数传入，在其中被调用并用于挂载，其详细功能将在后面描述。
- 7 此外，它还对超级块，位示图等结构进行了管理，将设备上的信息记录在内存。
- 8
- 9 而**dev**层则是具体的设备管理结构及其驱动的实现。在其中将向上提供**dop**操作由**vfs**或者**sfs**进行对接（不同设备间有差异，对于允许挂载的**disk0**，提供**dop**，与**vfs**和**sfs**对接，而对于不允许挂载的**stdin**，**stdout**等，则应当与**vfs**对接）
- 10 向下，其调用**d_op**系列函数，这些函数可以被理解为具体的设备驱动函数，实现细节不是**fs**实现过程中需要太关心的。

- 文件系统初始化过程：

在本次实验中，我们使用**fs_init**对文件系统进行同一初始化，在其中先后对**vfs**、**devs**（**disk0**，**stdin**，**stdout**）、**sfs**进行了初始化，这个先后顺序是严格的。

在**vfs**的初始化中，我们主要对其互斥访问的信号量**bootfs_sem**进行了初始化，并对设备表进行了初始化。

在**dev_init**中，我们先后对三种设备进行了初始化，在其中，将调用其各自的设备节点初始化函数，如**dev_init_disk0**，这个函数中将初始化一个**inode**用于记录设备，然后设置其**in_op**等字段，保证**vfs**能够找到其对应的操作。

在**sfs_init**中，我们做的是将**sfs**挂载到**disk0**上的工作，在**sfs_do_mount**中，我们会加载设备的超级块和位示图到内存，设置文件系统的函数指针，并将当前**fs**的指针写到参数**fs_store**所指，以便后续将其写入**vdev->fs**

- 用户程序的加载过程

在本次实验中，我们对**do_execve**进行了重写，由以前的从内存中加载程序变为从文件系统中加载程序。

在本次**do_execve**的实现过程中，主要的难点是将**argc**和**argv**写入用户栈以及将用户程序从**fs**抄到内存空间的过程。

我们将使用参数中的第一个作为文件的**path**调用**sysfile_open**对其进行打开得到其**fd**，这个打开过程我们将在后面详述。

然后后调用其**load_icode**，将参数及文件内容写入虚拟内存空间。我们会依序将**argv**、**pointer(argv)**、**argc**、**user_sp**布局在虚拟内存空间的高地址位置。

然后循环地将文件系统中的**elf**文件一页一页地抄到**va**所指向的位置，这个过程的实现与上次有所区别，但核心思想和基本功能一致，我们不再赘述。

- **open**的过程

在**open**函数中，我们首先通过用户态的系统调用**ecall**，经中断的处理一步步调用到**sysfile_open**，这中间的过程较为简单，我们省略。

在**sys_file_open**中，我们会先将**path**复制到内核空间，然后调用**file_open**在进程的文件描述符表中为其分配一个空闲槽以供使用。

在vfs_open中，我们对不同文件系统的文件打开统一处理：最主要的打开逻辑在vfd_lookup中，这里会对路径进行处理：首先拿到路径所属的device，然后拿到device对应的文件系统，并将文件系统的根的inode回传，此后即可做vop_look，查找对应相对目录下的文件。

- 文件存储空间的管理策略

目前的操作系统中，主流的空闲磁盘块管理策略有四种，分别为空闲表法、空闲链表法、位示图法和成组链接法。在uCore中的实现方式是位示图法，我们对另外三种方法进行简述，对位示图法进行详述。

- 1 空闲表法：以管理空闲区间的的方式管理空闲块。
- 2 一个空闲区间由一或多个连续的空闲块组成，在空闲表中，以一个二元组（一个空闲区间的第一个空闲块号，该区间的空闲块数）为表项，记录了空闲块的信息。
- 3 这样的管理下，空闲磁盘块的分配方式与内存的动态分区分配相似。
- 4
- 5 空闲链表法：依管理单元的不同，分为两种管理方式，分别为空闲盘块链法和空闲盘区链法，其实现大同小异。
- 6 使用链表管理，其管理节点的表项与空闲表法管理的表项类似。
- 7 额外地，空闲盘区链的管理方式需要考虑空闲块回收时的合并问题。
- 8
- 9 成组链接法：这种方式适用于大型文件系统，如UNIX，缓和了大型文件系统中空闲表过大的问题。
- 10 在文件卷目录区，我们使用一整个磁盘块作为超级块（注意，此超级块与实验代码中的超级块是两个概念），用于存储下一组空闲盘块数和下一组中每一块的块号。
- 11 在每一块的第一块，又要进一步存储下一块的对应信息。这样的记录方式会有相对复杂的分配与回收方式，我们略去。
- 12
- 13 位示图法：我们使用 字数行*字长列 布局的二进制表记录每一块的分配信息。
- 14 这么说有些抽象，具体到uCore中，我们以4B,32b为字长，那么我们的表列数即为32列。
- 15 而字数将由构造bitmap时传入的参数nbits决定。这个参数的语义是create出来的位示图至少要能示明这么多位的占用信息。
- 16 但是当nbits的传入参数不是字长的整数倍时，为方便管理，我们需要使用变量 `nwords=ROUNNUP_DIV(nbits,word_bits)` 作为字数，即行数。
- 17 如：当我们使用63作为nbit时，不足两行，补齐为两行，那么nwords即为2（`ROUNDUP（63，32）/32`）。
- 18 而对于这些补齐的位，我们需要在最开始就将其置零，表示已占用，避免拿到根本不存在的块。
- 19 值得一提的是：对于不同的系统，位示图中的01标志含义有所差异，有的以0为占用，有的以1为占用，这无伤大雅。我们的uCore中使用的方式是前者。